

Assignment_10.2

2303A510H5

Batch_30

Task Description -1(Error Detection and Correction)

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
```

```
sum = 0
```

```
for n in nums
```

```
sum += n
```

```
return total
```

```
# AI to analyze a Python script and correct all syntax and logical errors.
#def calculate_total(nums)
#sum = 0
#for n in nums
#sum += n
#return total
#Give Corrected and executable Python code with brief explanations of the identified syntax and logic errors.
def calculate_total(nums):
    # Initialize the sum variable to store the total
    total = 0
    # Iterate through each number in the input list
    for n in nums:
        # Add the current number to the total
        total += n
    # Return the final total
    return total
# Explanation of identified errors:
# 1. The function definition was missing a colon (:) at the end of the line. This is required to indicate the start of the function body.
# 2. The variable 'sum' was used to store the total, but it is a built-in function in Python. It is better to use a different variable name (e.g., 'total') to avoid confusion and potential conflicts.
# 3. The for loop was missing a colon (:) at the end of the line, which is necessary to indicate the start of the loop body.
# 4. The return statement was returning 'total', but it was not defined in the original code. It should return the variable that was used to accumulate the sum (in this case, 'total').]
```

Justification:

The original code had syntax errors (missing colons) and a logical mistake (returning an undefined variable). Correcting these ensures the function runs properly and consistently returns the intended sum.

Task Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

```
def findSum(a,b):return a+b
```

```
print(findSum(5,10))
```

code:

```
#Use AI to refactor Python code to comply with standard coding style guidelines.
#def findSum(a,b):return a+b
#print(findSum(5,10))
#Give Well-structured, consistently formatted Python code following standard style conventions.
def find_sum(a, b):
    """
    Returns the sum of two numbers.

    Args:
        a (int): The first number.
        b (int): The second number.

    Returns:
        int: The sum of the two numbers.
    """
    return a + b
# Example usage
print(find_sum(5, 10))
# Explanation of refactoring:
# 1. The function name was changed from 'findSum' to 'find_sum'
#    to follow the standard naming convention for functions in Python (snake_case).
# 2. A docstring was added to provide a clear description of the function's purpose, its parameters, and its return value. This enhances code readability
#    and maintainability.
# 3. The code was formatted to be more readable, with proper indentation and spacing. This helps improve the overall structure and clarity of the code.
```

Output:

```
PS C:\Users\parim\Ai Assistant coding> & C:/Users/parim/Python/Scripts/Python3\python.exe C:\Users\parim\Ai Assistant coding\find_sum.py
15
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

Refactoring to PEP 8 standards (snake_case, spacing, multi-line structure) makes the code easier to read, maintain, and share with others, aligning with professional Python practices.

Task Description -3(Code Clarity Improvement)

Task:

Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):  
  
    return x-y*2  
  
print(f(10,3))
```

code:

```
#Use AI to improve code readability without changing its functionality.  
#def f(x,y):  
#return x-y*2  
#print(f(10,3))  
#guve Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.  
def calculate_difference(x, y):  
    """  
    Calculates the difference between two numbers, where the second number is multiplied by 2.  
  
    Args:  
        x (int): The first number.  
        y (int): The second number.  
  
    Returns:  
        int: The result of x minus twice y.  
    """  
    return x - (y * 2)  
# Example usage  
print(calculate_difference(10, 3))  
# Explanation of improvements:  
# 1. The function name was changed from 'f' to 'calculate_difference' to  
#    provide a clear and descriptive name that indicates the purpose of the function.  
# 2. The variable names 'x' and 'y' were retained as they are  
#    commonly used for mathematical operations, but the function's docstring provides context for their meaning.  
# 3. A docstring was added to explain the function's purpose, its parameters,  
#    and its return value, which enhances code readability and maintainability.  
# 4. Proper indentation and spacing were applied to improve the overall structure and clarity of the  
#    code, making it easier to read and understand.
```

Output:

```
4  
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

Using meaningful names for functions and variables clarifies the purpose of the code. This improves readability and makes it easier for others (or your future self) to understand without guessing.

Task Description -4(Structural Refactoring)

Task:

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

```
print("Hello Ram")
```

```
print("Hello Sita")
```

```
print("Hello Ravi")
```

code:

```
# Use AI to refactor repetitive code into reusable functions.
#print("Hello Ram")
#print("Hello Sita")
#print("Hello Ravi")
def greet(name):
    """
    Prints a greeting message for the given name.

    Args:
        name (str): The name of the person to greet.
    """
    print(f"Hello {name}")
# Example usage
greet("Ram")
greet("Sita")
greet("Ravi")
# Explanation of refactoring:
# 1. A reusable function named 'greet' was created to encapsulate the logic for printing a greeting message.
# 2. This eliminates the need for repetitive code and promotes code reusability.
# 3. The function takes a single parameter 'name', which allows it to be flexible
#    and adaptable to different inputs, making it more versatile.
# 4. A docstring was added to provide a clear description of the function's purpose
#    and its parameter, enhancing code readability and maintainability.
# 5. The example usage demonstrates how to call the 'greet' function with different
#    names, showcasing the benefits of refactoring the code into a reusable function.
```

Output:

```
7.2/ass.10.2.py  
Hello Ram  
Hello Sita  
Hello Ravi  
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

By introducing a reusable function, repetitive print statements are eliminated. This modular approach reduces redundancy, improves maintainability, and allows easy scaling if more names need greetings.

Task Description -5(Efficiency Enhancement)

Task:

Use AI to optimize Python code for better performance.

Sample Input Code:

```
numbers = [ ]  
  
for i in range(1, 500000):  
    numbers.append(i * i)  
  
print(len(numbers))
```

code:

```
#Use AI to optimize Python code for better performance.
#numbers = [ ]
#for i in range(1, 500000):
#    numbers.append(i * i)
#print(len(numbers))
squares = [i * i for i in range(1, 500000)]
print(len(squares))
# Explanation of optimization:
# 1. The original code uses a for loop to iterate through a range of numbers and
#    appends the square of each number to a list. This approach can be inefficient
#    due to the overhead of multiple append operations.
# 2. The optimized code uses a list comprehension, which is a more concise and efficient way to create a list.
#    List comprehensions are generally faster than traditional for loops for creating lists because they are optimized
#    in Python and reduce the overhead of function calls and multiple append operations.
# 3. The list comprehension allows for the creation of the entire list of squares in a single line of code, improving
#    readability and performance.
```

Output:

```
7.2/ass.10.2.py"
499999
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

Switching from a loop with `.append()` to a list comprehension improves performance because comprehensions are optimized internally. The result is the same, but execution is faster and more Pythonic.